

数据结构与算法设计

第一章 绪论

本章要求与重难点

- 熟练掌握数据、数据结构等基本概念（重点）
- 熟练数据结构的三个方面的内容（重点）
- 理解抽象数据类型的概念及表示（难点）
- 理解算法要素的确切含义，掌握算法设计的基本要求以及分析算法的时间与空间复杂度的方法。（重点与难点）

1.1 什么是数据结构

数据结构的创始人——高德纳



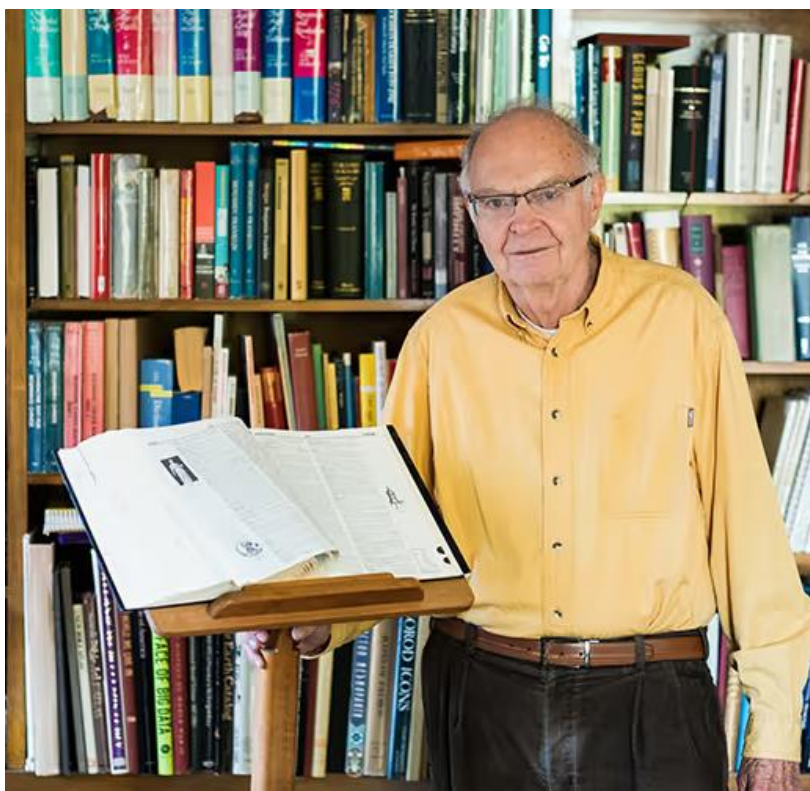
Donald E. Knuth (

1938年出生，25岁毕业于加州理工学院数学系，博士毕业后留校任教，28岁任副教授。30岁时，加盟斯坦福大学计算机系，任教授。从31岁起，开始出版他的历史性经典巨著：

The Art of Computer Programming

他计划共写7卷，然而出版三卷之后，已震惊世界，使他获得计算机科学界的最高荣誉图灵奖，此时，他年仅36岁。

计算机科学泰斗Donald E. Knuth（高德纳）



《计算机程序设计艺术》

——计算机领域经典巨著

首次提出

“算法” (Algorithm)

“数据结构” (Data Structure)



程序的概念

- 什么是程序？

- 程序是用某种计算机能理解并执行的计算机语言描述解决问题的方法步骤。

- ① 程序的本质是什么？

- **数据表示**：将数据存储在计算机中
- **数据处理**：处理数据，求解问题

数据结构问题起源于程序设计

数据结构的起源

- 计算机求解问题:

问题 → 抽象出问题的模型 → 求模型的解

- 问题——数值问题、非数值问题

☞ 数值问题

- 程序处理的数据是纯粹的数值，数据之间的关系主要是数学方程或数学模型等。

☞ 非数值问题

- 处理多种复杂的具有一定结构关系的数据。
- 数据元素之间的相互关系一般无法用数学方程加以描述，而是要用线性表、树、图等**数据结构**来描述。

1.1 什么是数据结构?

问题1：图书检索自动化

- 图书的基本信息：书名，作者，分类号，出版单位，出版时间
作者简介，内容简介等等。
- 操作：检索，排序，等等
- 数据之间的关系：线性关系
- 数据表示和算法操作的设计：与需求有关

1.1 什么是数据结构?

线性表

书目文件

	书 号	书 名	作 者	出版时间	出版社
001	L001	高等数学	樊映川	1964	高教出版社
002	S002	理论力学	罗远祥	1965	高教出版社
003	M003	化学	张之	1982	清华大学出版社
004	L002	线性代数	栾汝书	1958	清华大学出版社
005	K005	运筹学	刘永年	1979	清华大学出版社
006	L003	...	...	...	...

索引表

按书名

高等数学	001
理论力学	002
化学	003
线性代数	004
...	...

按作者名

樊映川	001
罗远祥	002
张之	003
栾汝书	004
...	...

按分类

L	001,004,006
K	005
M	003
S	002
...	...

书目信息

机读格式(MARC)

题名/责任者: 计算机程序设计艺术.第1卷.基本算法/(美) Donald E. Knuth著 苏运霖译

出版发行项: 北京:国防工业出版社,2002

ISBN及定价: 7-118-02799-5 精装/CNY98.00

载体形态项: xix, 626页:图;24cm

并列正题名: Art of computer programming. Vol. 1. Fundamental algorithms

其它题名: 基本算法

个人责任者: 克努特, D. E. (Knuth, Donald E.) 著

个人次要责任者: 苏运霖 译

学科主题: 电子计算机-程序设计

学科主题: 电子计算机-算法设计

中图法分类号: TP311

一般附注: 经典计算机科学著作最新修订版

出版发行附注: 本书据Addison Wesley Longman 1997年英文第3版译出

相关题名附注: 英文并列题名取自版权页

责任者附注: 责任者规范汉译姓: 克努特

书目附注: 有索引



放入暂存书架

查看暂存书架(0)

收藏

馆藏信息

预约申请

委托申请

参考书架

图书评论

相关借阅

相关收藏

索书号	条码号	年卷期	馆藏地	书刊状态
TP311/ZK281/1	01698164	2003.4 -	嘉定密集书库	可借
TP311/ZK281/1	01698167	2003.4 -	嘉定密集书库	可借
TP311/ZK281/1	01698171	2003.4 -	嘉定密集书库	可借
TP311/ZK281/1	01735661	2003.4 -	嘉定密集书库	可借
TP311/ZK281/1	01698166	2003.4 -	嘉定保存本书库	可借
TP311/ZK281/1	01698162	2003.4 -	嘉定校区图书馆(中文) 九楼  定位	可借
TP311/ZK281/1	01698170	2003.4 -	嘉定校区图书馆(中文) 九楼  定位	可借
TP311/ZK281/1	01698163	2003.4 -	沪西密集书库	可借
TP311/ZK281/1	01698168	2003.4 -	沪西图书阅览室	非可借

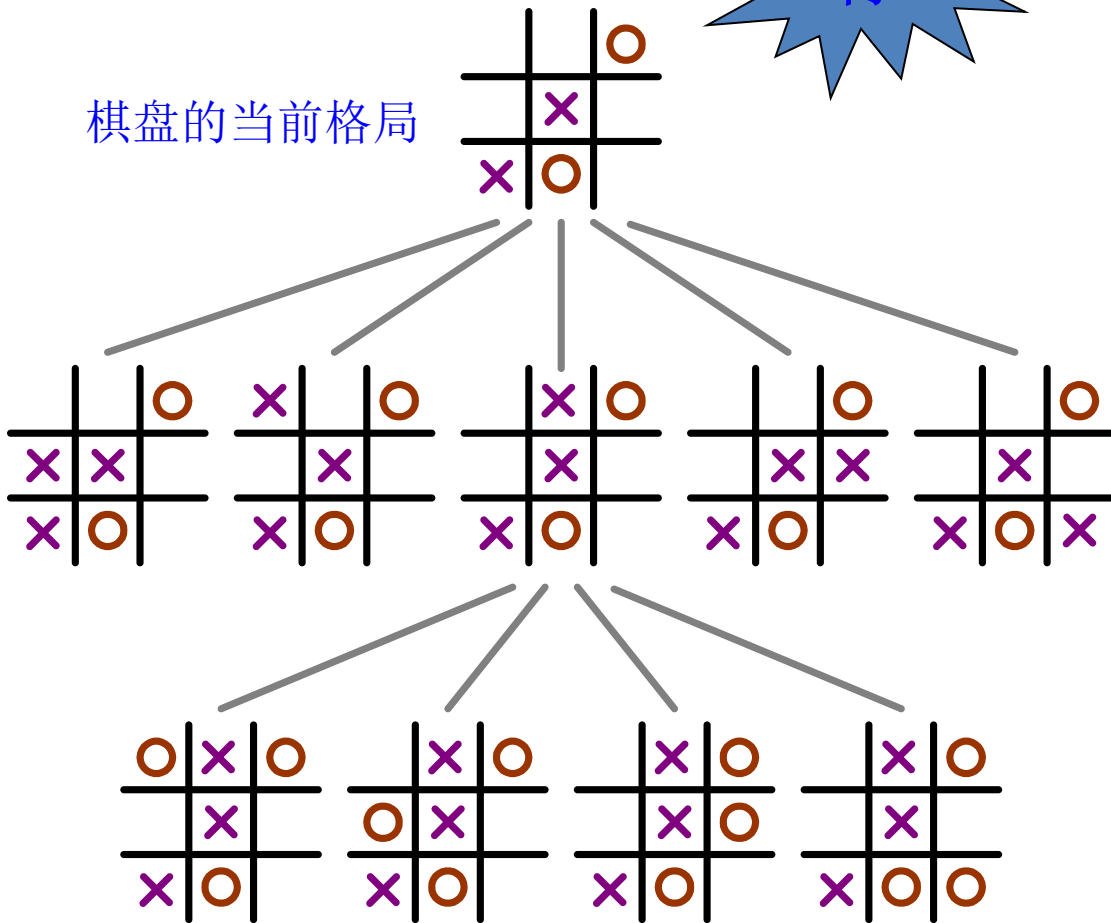
1.1 什么是数据结构?

树

问题2：井字棋

- 好的棋手不仅要看当前的格局，还要预测棋局发生的趋势，甚至最后的胜负结局
- 如何表示一个棋局
- 数据的逻辑结构：表示棋局之间的演化关系：树型结构
- 算法如何设计
基于数据表示的基础上算法设计

棋盘的当前格局



棋盘的对弈树局部

《数据结构》课程的发展历史及课程的重要性

❑ 起源于程序设计, 1968年在美国开设。它随着大型程序的出现而出现。

❑ 我国1980's年代初开设

学习《数据结构》的目的:

- ①提高复杂程序设计及软件开发的能力
- ②培养算法设计能力
- ③为后继课程（如操作系统、编译原理等）打基础。

课程意义

- **数据结构是计算机专业的一门专业基础课。**
- **数据结构是关于数据组织和处理的基本技术的一门学科。**
- **程序 = 数据结构 + 算法 + 文档**
- **数据结构是一门实践性很强的课程。**
- **重要的求职敲门砖**
- **重要的考研专业课程**

1.1 什么是数据结构？

- **简单说，数据结构是以某种方式联系在一起的数据元素的集合。程序中的数据结构反映了程序员在程序中表示信息的方法，算法反映了如何处理信息的方法。**
- **数据结构研究的是数据元素之间抽象化的相互关系及这种关系在计算机中的存储表示，对每种结构定义各自的运算，设计出相应的算法，并用某种语言实现该算法。**
- **数据结构的地位：数学、硬件、软件之间。核心专业基础课。**

对于一个课题，在计算机领域一般遵循下面的解决程序：

总体设计——模块分割——建立数学模型——解数

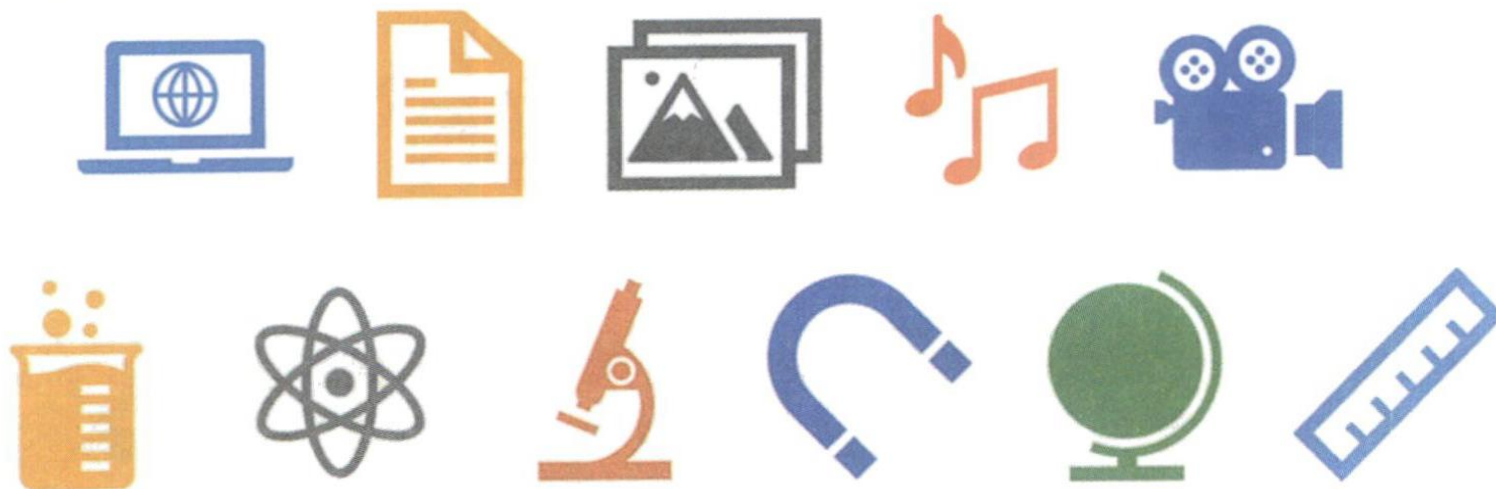
学模型的算法——程序编制——调试——结果

思考：计算机不能做什么？

1.2 基本概念和术语

1.2 基本概念和术语

数据

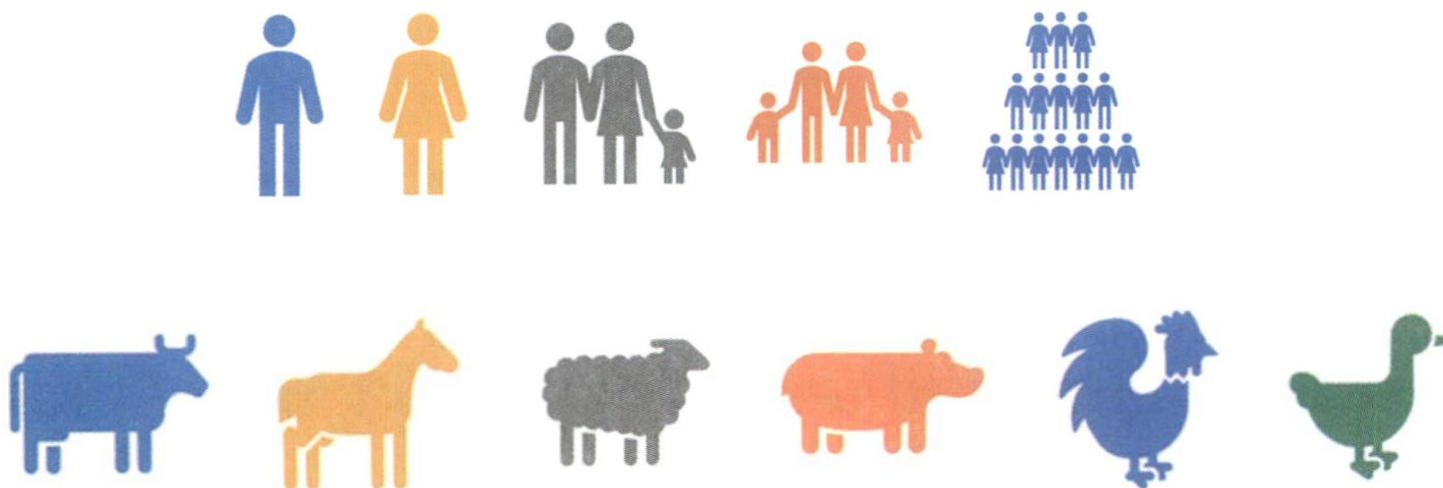


描述客观事物的符号，是计算机中可以操作的对象，是能被计算机识别，并输入给计算机处理的符号集合。

1.2 基本概念和术语

数据元素

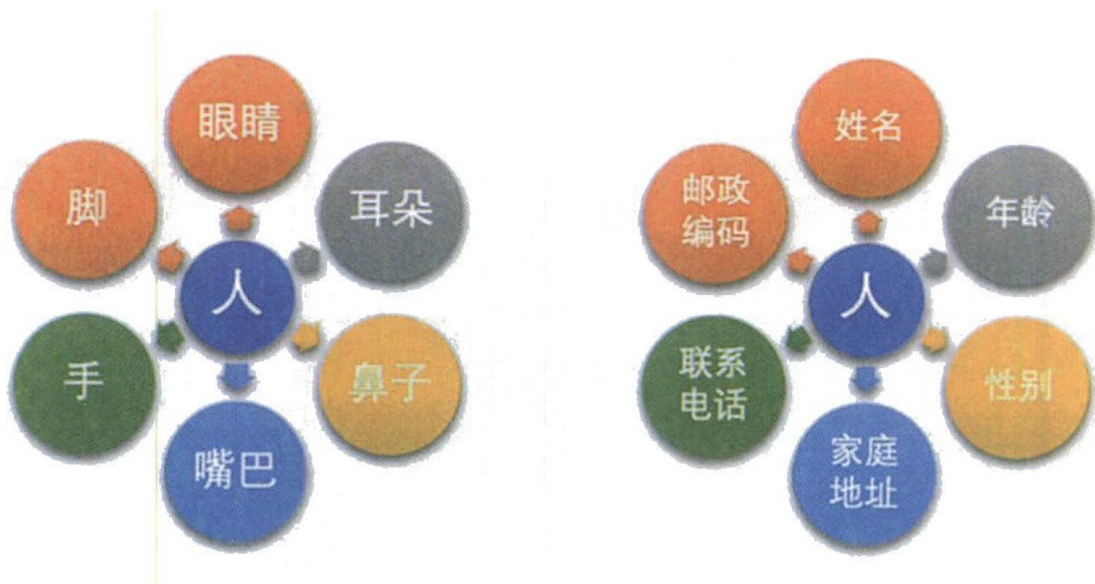
是组成数据的、有一定意义的基本单位，在计算机中通常作为整体处理，也被称为记录。



1.2 基本概念和术语

数据项

是数据不可分割的最小单位，一个数据元素可以由若干个数据项组成。



1.2 基本概念和术语

数据对象

是性质相同的数据元素的集合，是数据的子集。

数据项

数据元素

学号	姓名	班号	性别	出生日期	入学成绩
001	刘影	01	女	19840417	523
002	李恒	01	男	19831211	679
003	陈诚	02	男	19840910	638
004	...	...	...	...	...

整个表是学生成绩数据对象

1.2 基本概念和术语

数据结构

定义：是相互之间存在一种或多种特定关系的数据元素的集合。

数据之间不是相互独立的，他们之间有某种特定的关系，这种数据元素之间的关系，称为“结构”。

结构=关系+实体

- 另一种定义：按照逻辑关系组织起来的一批数据，按一定的存储方法把它存储在计算机中，并在这些数据上定义了一个运算的集合。
- 形式定义：二元组 (D, S) 其中 D 是数据元素的有限集， S 是 D

1.2 基本概念和术语

逻辑结构

数据元素之间的逻辑关系，与计算机无关。

可用一个二元组表示：Data_Structure = (D, R)

D：数据元素的有穷集合，R：集合D上关系的有穷集合。

[例] 设有数据结构 $B = (D, R)$,

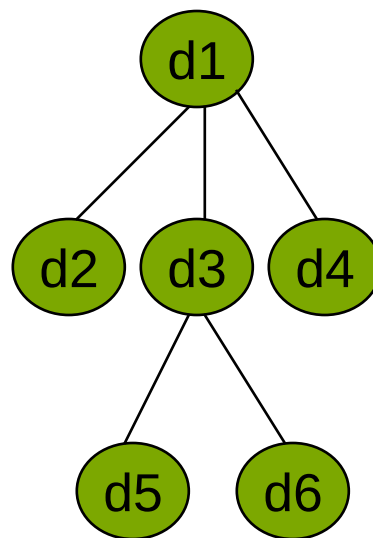
其中 $D = \{d1, d2, d3, d4, d5, d6\}$,

$R = \{r\}$,

$r = \{\langle d1, d2 \rangle, \langle d1, d3 \rangle,$

$\langle d1, d4 \rangle, \langle d3, d5 \rangle, \langle d3, d6 \rangle\}$,

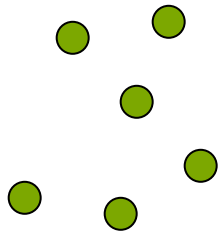
试画出其逻辑结构图。



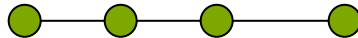
1.2 基本概念和术语

逻辑结构

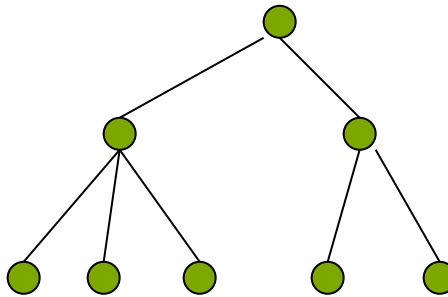
- (1) **集合结构** 数据元素除了“属于同一集合”的联系之外，没有其它的关系。
- (2) **线性结构** 数据元素之间存在一对一的关系。
- (3) **树型结构** 数据元素之间存在一对多的关系。
- (4) **图状结构或网状结构** 数据元素之间存在多对多的关系。



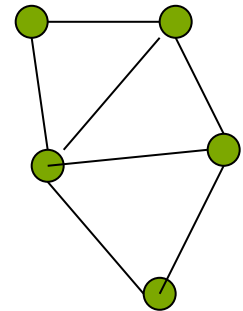
成员关系



长幼关系



管理关系



朋友关系

示例 1

- 用图形表示下列数据结构，并指出它们是属于线性结构还是非线性结构。

$S = (D, R)$

$D = \{ a, b, c, d, e, f \}$

$R = \{(a,e), (b,c), (c,a), (e,f), (f,d)\}$

- 解：上述表达式可用图形表示为：

b — **c** — **a** — **e** — **f** — **d**

此结构为线性的。

示例 2

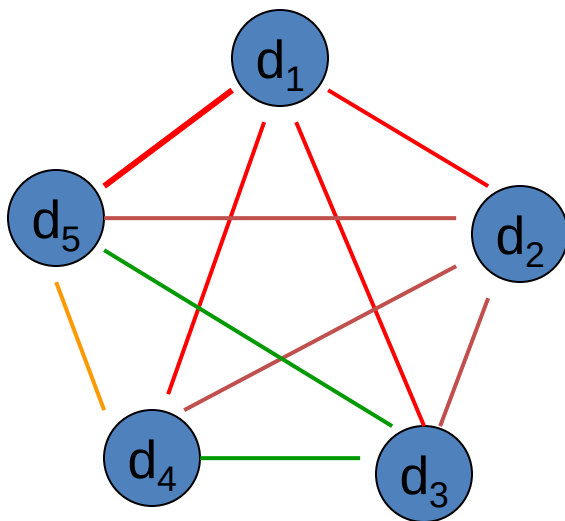
- 用图形表示下列数据结构，并指出它们是属于线性结构还是非线性结构。

$$S = (D, R)$$

$$D = \{d_i \mid 1 \leq i \leq 5\}$$

$$R = \{(d_i, d_j), i < j\}$$

解：上述表达式可用图形表示为：



该结构是非线性的。

1.2 基本概念和术语

存储结构

□ **存储结构**：数据的逻辑结构在计算机中如何表示。

■ **数据元素的映象**

用二进制位(bit)的位串表示数据元素。

每个数据元素的映象称为**结点**

每个数据项的映象称为**数据域**

■ **关系的映象**

两种基本方法及其组合使用。

● **顺序映象**：以相对的存储位置表示关系→**顺序存储结构**

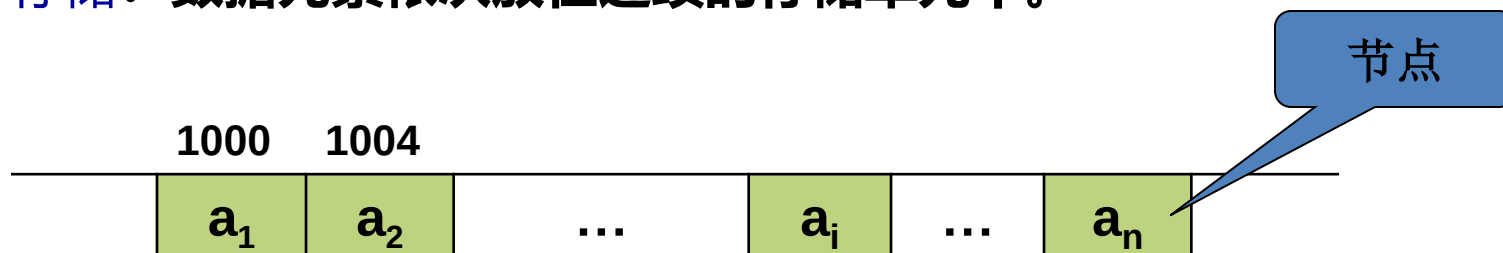
● **链式映象**：以附加信息(指针)表示关系→**链式存储结构**

在不同的编程环境下，存储结构有不同的描述方式。

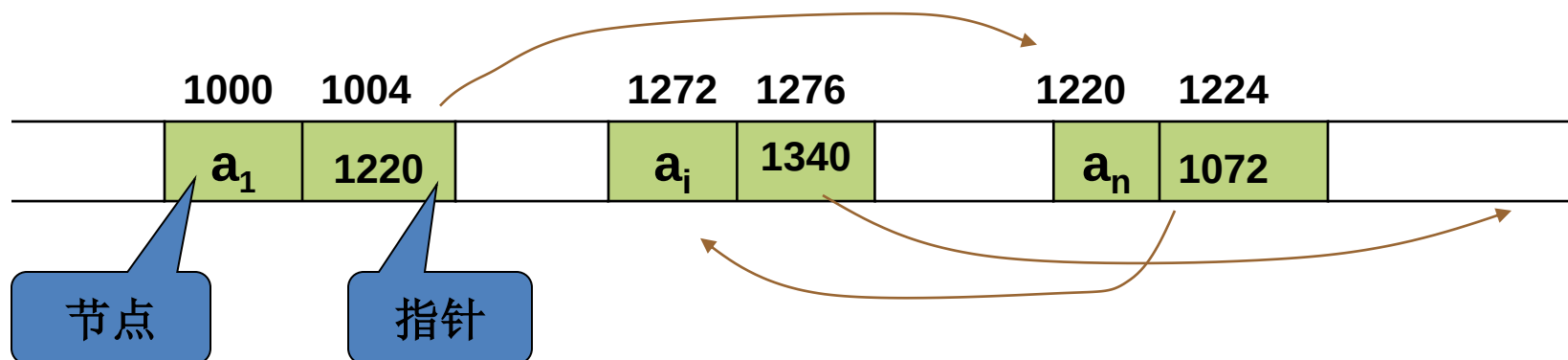
用高级程序语言编程时，直接用其提供的数据类型描述。

顺序存储和链式存储

(1) **顺序存储**：数据元素依次放在连续的存储单元中。



(2) **链式存储**：在存储结点中增加若干指针域，记录后继或者相关结点的地址（指针）。



1.2 基本概念和术语

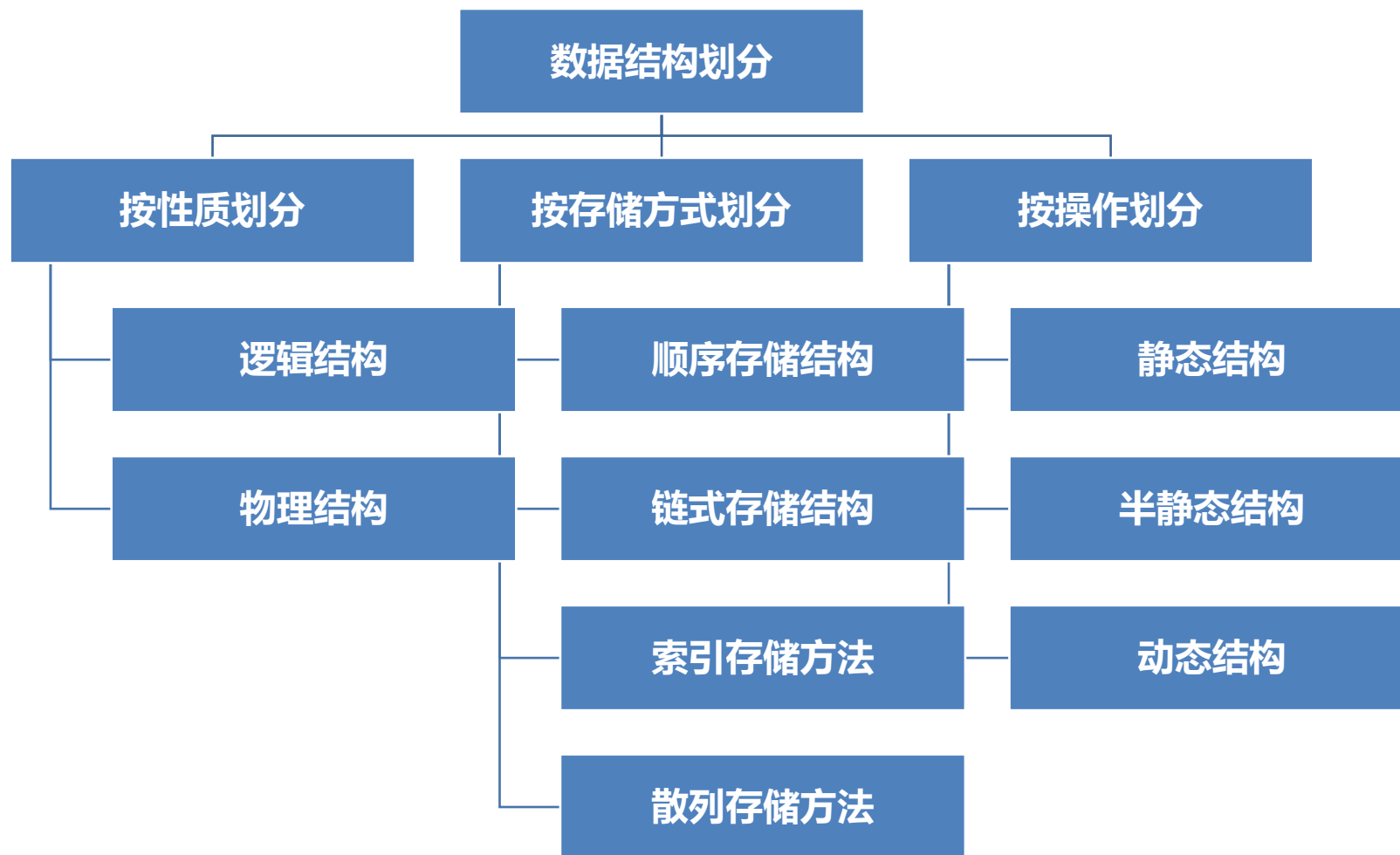
数据运算

- **运算（操作）**：在数据逻辑结构上定义的一组数据被使用的方式，其具体实现要在存储结构上进行。
- 常用的运算有：
 - (1)建立数据结构
 - (2)清除数据结构
 - (3)插入数据元素
 - (4)删除数据元素
 - (5)排序
 - (6)检索*
 - (7)更新
 - (8)判空和判满*
 - (9)求长*
- 操作为**引用型操作**，即数据值不发生变化；其它为**加工型操作**。

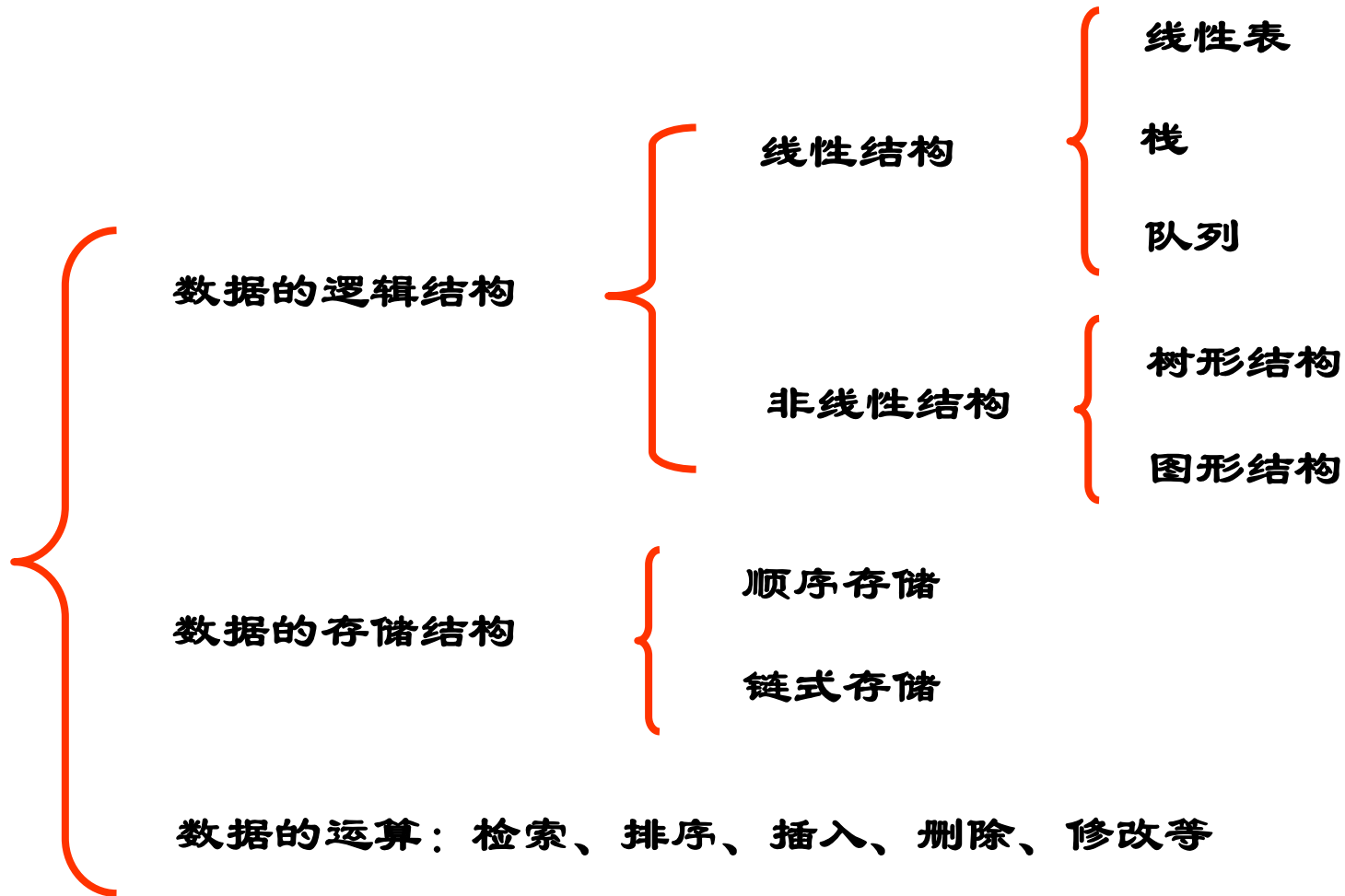
1.2 基本概念和术语

- **数据的逻辑结构独立于计算机，是数据本身所固有的。**
- **存储结构是逻辑结构在计算机存贮器中的映像，必须依赖于计算机。**
- **运算是指所施加的一组操作总称。运算的定义直接依赖于逻辑结构，但运算的实现必须依赖于存贮结构。**

1.2 基本概念和术语



1.2 基本概念和术语



1.3 数据类型和抽象数据类型

抽象数据类型

- **数据类型**：是一个**值的集合**和定义在这个值集上**一组操作**总称。
- **抽象数据类型 ADT (Abstract Data Type)**：是指基于一个逻辑类型的数据模型以及定义在该模型上的一组操作。每一个操作由它的输入和输出定义。
- **分类**：
 - **原子类型**：是不可以再分解的基本类型，包括整型、实型、字符型等。
 - **结构类型**：由若干个类型组合而成，是可以再分解的。例如，整型数组是由若干整型数据组成的。

C语言相关知识回顾



抽象数据类型

- **形式：三元组 (D, S, P) S 是 D 上的关系集， P 是对 D 的基本操作集**
- **说明：一个ADT的定义并不涉及它的实现细节。这些实现细节对于ADT的用户是隐藏的。隐藏实现细节的过程称为封装。数据结构是ADT的物理实现。ADT的每一个操作均由一个或多个子程序来实现。**

ADT抽象数据类型名{

 数据对象：<数据对象的定义>

 数据关系：<数据关系的定义>

 基本操作：<基本操作的定义>

}ADT抽象数据类型名

抽象数据类型

- 例：线性表这样的抽象数据类型，其数学模型是：数据元素的集合，该集合内的元素有这样的关系：除第一个和最后一个外，每个元素有唯一的前趋和唯一的后继。可以有这样一些操作：插入一个元素、删除一个元素等。

名称	线性表	解释
数据对象	$D=\{a_i \mid a_i(-ElemSet, i=1,2,...,n, n \geq 0)\}$	任意数据元素的集合
数据关系	$R1=\{<a_{i-1}, a_i> \mid a_{i-1}, a_i(-D, i=2,...,n)\}$	除第一个和最后一个外，每个元素有唯一的直接前趋和唯一的直接后继
基本操作	ListInsert(L,i,e)	L为线性表，i为位置，e为数据元素。
	ListDelete(L,i,e)	
	...	

抽象数据类型

- **作用：**抽象数据类型可以使我们更容易描述实际问题。
- **好处：**可提高软件的复用程度。使用它的人可以只关心它的逻辑特征，不需要了解它的存储方式。定义它的人同样不必要关心它如何存储。

实践作业举例—复数运算

□ 抽象数据类型复数定义:

ADT Complex

{

数据对象: $D = \{e1, e2 | e1, e2 \text{ 均为实数} \}$

数据关系: $R1 = \{ \langle e1, e2 \rangle | e1 \text{ 是复数的实数部分, } e2 \text{ 是复数的虚数部分} \}$

基本运算:

AssignComplex(&Z, v1, v2) //构造复数Z, 其实部和虚部分别赋以
//参数v1和v2的值

DestroyComplex(&Z) //复数Z被销毁

GetReal(Z, &real) //用real返回复数Z的实部值

GetImag(Z, &imag) //用imag返回复数Z的虚部值

Add(z1, z2, &sum) //用sum返回两个复数z1、z2的和值

}ADT Complex

实践作业举例—复数运算

□ 复数运算实现

```
#include<stdio.h>
```

```
typedef struct{
```

```
    float e1;
```

```
    float e2;
```

```
} Complex;
```

```
void AssignComplex(Complex &Z, float v1, float v2)
```

```
{
```

```
    Z.e1=v1; Z.e2=v2;
```

```
}
```


实践作业举例—复数运算

```
void GetReal(Complex Z, float &real)  
{    real=Z.e1; }
```

```
void GetImag(Complex Z, float &imag)  
{    imag=Z.e2; }
```

```
void Add(Complex z1, Complex z2, Complex &sum)  
{  
    sum.e1=z1.e1+z2.e1;  
    sum.e2=z1.e2+z2.e2;  
}
```

实践作业举例—复数运算

```
void main()
```

```
{
```

```
    Complex X,Y,Z; float x1,x2,y1,y2,a1,a2,b1,b2;
```

```
    printf( "Please input complex X 's real and imag:" );
```

```
    scanf( "%f%f" , &x1, &x2);
```

```
    printf( "You input the complex X is %f +%f i" , x1, x2);
```

```
    AssignComplex(X,x1,x2);
```

```
    printf( "Please input complex Y 's real and imag:" );
```

```
    scanf( "%f%f" , &y1, &y2);
```

```
    printf( "You input the complex Y is %f +%f i" , y1, y2);
```

```
    AssignComplex(Y,y1,y2);
```

实践作业举例—复数运算

```
GetReal(X,a1); GetImag(X,a2);  
printf(“The complex X’s real is %f , X’s image is  
%f .”,a1,a2);  
GetReal(Y,b1); GetImag(Y,b2);  
printf(“The complex Y’s real is %f , Y’s image is  
%f .”,b1,b2);  
Add(X,Y,Z);  
GetReal(Z,a1); GetImag(Z,a2);  
printf(“The complex Z’s real is %f , Z’s image is  
%f .”,a1,a2);
```

```
}
```

回顾

□ 研究和解决非数值数据的组织和处理

- 描述非数值计算问题的数学模型，不再是数学方程
- 例如：前述的三个例子：数据的线性结构，树型结构，图

□ 算法+数据结构=程序

- 算法和数据结构之间的关系
- 软件系统的框架应当建立在数据之上，而不是建立在操作之上

□ 数据结构的作用范畴

- 抽象数据对象的数学模型（逻辑结构） 例：图状结构
- 明确操作（运算的定义） 例：查找、插入、.....
- 在存储结构上映射数据（存储结构） 例：顺序存储
- 实现操作

1.4 算法和算法分析

1.4 算法和算法分析

1+2+3+.....+100

```
int i, sum = 0, n = 100;
for (i = 1; i <= n; i++)
{
    sum = sum + i;
}
printf(" %d ", sum);
```

1.4 算法和算法分析

$$\begin{aligned}\text{sum} &= 1 + 2 + 3 + \dots + 99 + 100 \\ \text{sum} &= 100 + 99 + 98 + \dots + 2 + 1 \\ 2 \times \text{sum} &= \underbrace{101 + 101 + 101 + \dots + 101 + 101}_{\text{共 } 100 \text{ 个}}\end{aligned}$$

所以 $\text{sum}=5050$

```
int i, sum = 0, n = 100;
sum = (1 + n) * n / 2;
printf("%d", sum);
```

1.4 算法和算法分析

1、算法的定义

指一系列确定的而且是在有限步骤内能完成的操作。

2、算法的五个特性

- (1) 有穷性 (能执行结束)
- (2) 确定性 (算法每一步骤都具有确定的含义, 不会出现二义性)
- (3) 0至多个输入
- (4) 1至多个输出
- (5) 有效性(可行性)

——用于描述算法的操作都是足够基本的

□问题：程序是不是算法？

程序不一定满足有穷性（死循环）；程序中的指令必须是机器可执行的，而算法中的指令则无此限制。一个算法若用计算机语言来书写，则它就可以是一个程序。

算法与数据结构的关系

3、算法与数据结构的关系

□ 计算机科学家沃斯（N. Wirth）：

“算法+数据结构=程序”

程序设计本质：对实际问题选择一种好的数据结构，加上设计一个好的算法，而好的算法很大程度上取决于描述实际问题的数据结构。算法与数据结构是互相依赖、互相联系的。

□ 一个算法总是建立在一定数据结构上的；反之，算法不确定，就无法决定如何构造数据。

例1：编写程序查询某城市某人的电话号码

建立一张登记表，存放2个数据项：姓名+Tel。好的算法取决于这张表的结构及存储方式。

（1）将表中结点按照姓名顺序地存储在计算机中，依次查找，可能遍历整个表都找不到。

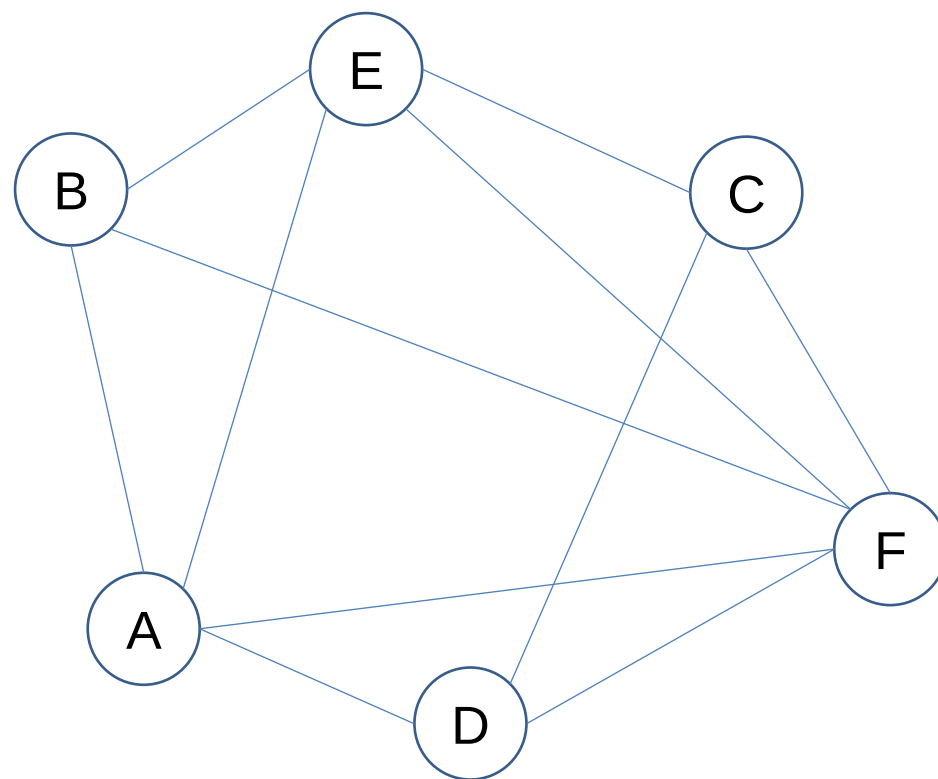
（2）再建立一张姓氏索引表，姓+表中的起始地址。不需查找其他姓氏。查找效率提高。

例2：设计一个考试日程安排表，使在尽可能短的时间内安排完考试，要求同一个学生选修的几门课程不能安排在一个时间内。

姓名	选修1	选修2	选修3
	A	B	E
	C	D	
	C	E	F
	D	F	A
	B	F	

- 解决该问题，首先选择一个合适的数据结构。用无向图表示，图中的顶点表示课程，不能同时考试的课程之间连上一条边。则该问题就抽象成对该无向图进行“着色”操作，即用尽可能少的颜色去给图中每个顶点着色，使得任意两个相邻的顶点着不同的颜色。同一种颜色表示一个考试时间。
- 解决问题的关键步骤是先选取合适的数据结构表示问题，才能写出有效的算法。

姓名	选修 1	选修 2	选修 3
	A	B	E
	C	D	
	C	E	F
	D	F	A
	B	F	



●答案： 1： A, C 2:B, D 3:E 4： F

算法设计的要求

4、算法设计的要求

□ 正确性(Correctness)

- 算法应满足具体问题的需求
- 对于典型的、苛刻而带有刁难性的一组有效输入得到正确的结果

□ 可读性(Readability)

- 算法应该好读。以有利于阅读者对程序的理解。

□ 健壮性(Robustness)

- 算法应具有容错处理。当输入非法数据时，算法应对其作出反应，而不是产生莫名其妙或随机的输出结果。

□ 高效性(Efficiency)

- **效率指的是算法执行时间。**对于解决同一问题的多个算法，执行时间短的算法效率高。
- 存储量需求指算法执行过程中所需要的最大存储空间。
- 时间复杂度和空间复杂度都与问题的规模有关。

算法效率的度量

- **定义：一个算法如果能在所要求的资源限制内将问题解决好，则称这个算法是有效率的。**
- **例如，一个资源限制是：可用来存储数据的全部空间**
 - **可能是分离的内存空间限制和磁盘空间限制**
 - **允许执行每一个子任务所需要的时间**

算法效率的度量

5、算法的分析 —— 算法性能的评价

□ 评价标准：

- 1) 算法所需的计算时间
- 2) 算法所需的存储空间
- 3) 算法的复杂性

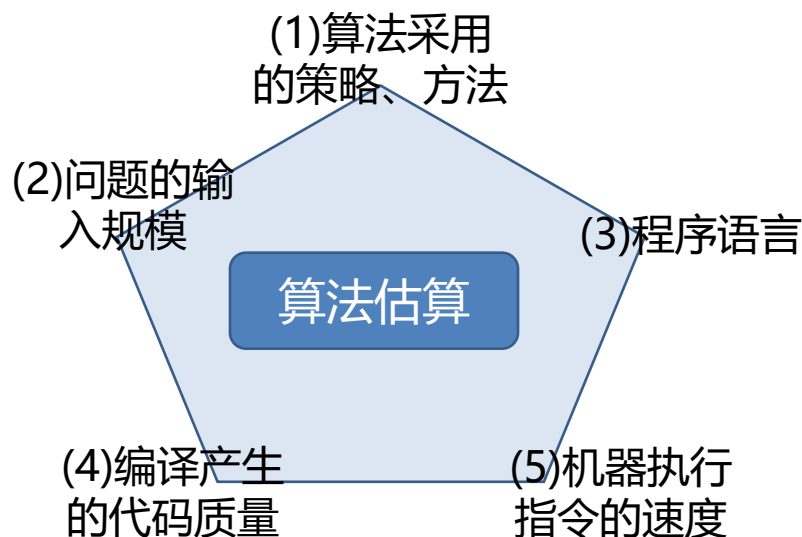
□ 度量算法执行时间的两种方法

1) 事后统计法

此方法有两个缺陷

2) 事前分析估算法

此方法取决于多个因素



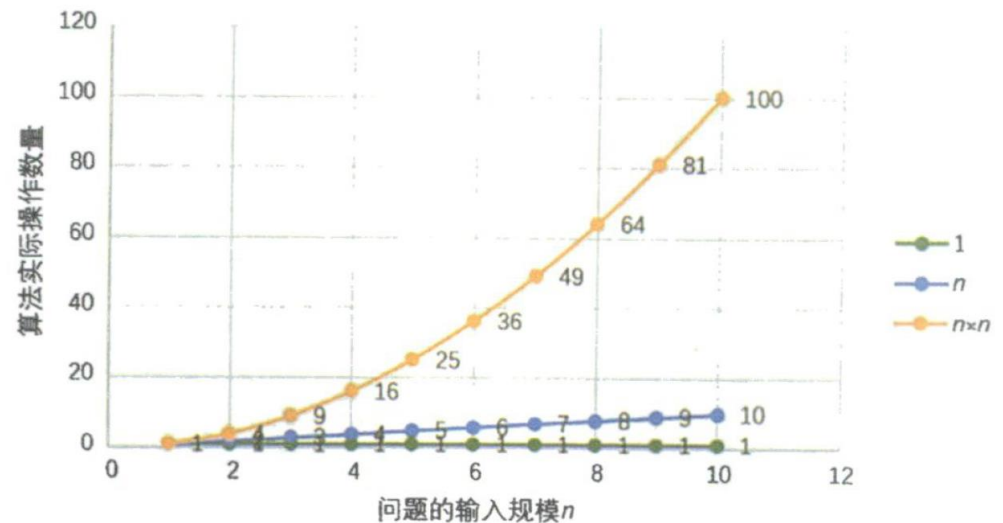
算法效率的度量

```
int i, sum = 0, n = 100;
for (i = 1; i <= n; i++)
{
    sum = sum + i;
}
printf ("%d", sum) ;
```

```
int sum = 0, n = 100;
sum = (1 + n) * n / 2;
printf ("%d", sum) ;
```

```
int i, j, x = 0, sum = 0, n = 100; /* 执行1次 */
for (i = 1; i <= n; i++)
{
    for (j = 1; j <= n; j++)
    {
        x++;
        sum = sum + x;
    }
}
printf ("%d", sum) ; /* 执行1次 */
```

不同算法的操作数量对比



算法效率的度量

6、时间复杂度

(1) 定义:

一般情况下, 算法中**基本操作重复执行**的时间是问题规模 n 的某个函数 $f(n)$, 算法的时间量度记作

$$T(n) = O(f(n))$$

它表示随问题规模 n 的增大, 算法执行时间的增长率和 $f(n)$ 的增长率相同, 称作算法的渐进时间复杂度, 简称时间复杂度。

- 语句的频度: 该语句重复执行的次数。

(2) 例

(a) `{ ++x; s=0 }` `++x` 的频度为1

(b) `for (i=1; i<=n; ++i) { ++x; s+=x; }` `++x`的频度为 n

(c) `for (j=1; j<=n; ++j)`

`for (k=1; k<=n; ++k) { ++x; s+=x; }` `++x`的频度为 n^2

算法效率的度量

次数	算法 A ($2n + 3$)	算法 A' ($2n$)	算法 B ($3n + 1$)	算法 B' ($3n$)
$n = 1$	5	2	4	3
$n = 2$	7	4	7	6
$n = 3$			10	9
$n = 10$	23	20	31	30
$n = 100$	203	200	301	300

常数不影响效率的评价

次数	算法 C ($4n+8$)	算法 C' (n)	算法 D ($2n^2+1$)	算法 D' (n^2)
$n = 1$	12	1	3	1
$n = 2$	16	2	9	4
$n = 3$	20	3	19	9
$n = 10$	48	10	201	100
$n = 100$	408	100	20 001	10 000
$n = 1000$	4 008	1 000	2 000 001	1 000 000

最高次项相乘的常数不影响效率的评价

算法效率的度量

次数	算法 E ($2n^2+3n+1$)	算法 E' (n^2)	算法 F ($2n^3+3n+1$)	算法 F' (n^3)
n = 1	6	1	6	1
n = 2	15	4	23	8
n = 3				27
n = 10	231	100	2 031	1 000
n = 100	20 301	10 000	2 000 301	1 000 000

最高次项相乘指数大的函数增值的快

次数	算法 G ($2n^2$)	算法 H ($3n+1$)	算法 I ($2n^2+3n+1$)
n = 1	2	4	6
n = 2	8	7	15
n = 5	50	16	66
n = 10	200	31	231
n = 100			20 301
n = 1,000	2 000 000	3 001	2 003 001
n = 10,000	200 000 000	30 001	200 030 001
n = 100,000	20 000 000 000	300 001	20 000 300 001
n = 1,000,000	2 000 000 000 000	3 000 001	200 000 3000 001

最高阶项是影响效率的核心

时间复杂度

□ 时间复杂度（大O运算规则）

规则1: $kf(n) = O(f(n))$ //大O忽略常数因子

规则2: if $f(n) = O(g(n))$ and $g(n) = O(h(n))$

then $f(n) = O(h(n))$ //传递性

规则3: $f(n) + g(n) = O(\max\{f(n), g(n)\})$

规则4: if $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$

then $f_1(n) * f_2(n) = O(g_1(n) * g_2(n))$

1. 用常数 1 取代运行时间中的所有加法常数。
 2. 在修改后的运行次数函数中，只保留最高阶项。
 3. 如果最高阶项存在且不是 1，则去除与这个项相乘的常数。
- 得到的结果就是大 O 阶。

时间复杂度计算

例1: $x++$; $s = 0$;

用频度法分析: 将 $x++$ 看成是基本操作, 语句频度为 1

$T(n)=2$

算法的时间复杂度: $O(1)$ ---常数阶

例2: $\text{for } (i = 0; i < n; i++) \{$ //执行 $n+1$ 次
 $x++$; //语句频度为: n
 $s += x$; //语句频度为: n
 $\}$

$T(n)=2n+n+1=3n+1$, 则时间复杂度为: $O(n)$

也可以这样考虑: 将 x 自增看成是基本操作, 则语句频度为: n 其时间复杂度为: $O(n)$, 即时间复杂度为线性阶。

时间复杂度计算

例3: `for(i=2;i<=n;++i)`
 `for(j=2;j<=i-1;++j)`
 `{++x;a[i, j]=x;}`

语句频度为:

$$\begin{aligned}1+2+3+\dots+n-2 &= (1+n-2) \times (n-2)/2 = (n-1)(n-2)/2 \\ &= (n^2-3n+2)/2\end{aligned}$$

∴时间复杂度为 $O(n^2)$, 即此算法的时间复杂度为**平方阶**。

时间复杂度计算

例4: 矩阵相乘 $C=A \times B$

```
for (i = 0; i < n; i++)           //n+1
    for (j = 0; j < n; j++) {      //n*(n+1)
        c[i][j] = 0;              //n^2
        for (k = 0; k < n; k++)    //n^2 *(n+1)
            c[i][j] += a[i][k] * b[k][j]; //n^3
    }
```

$$T(n) = 2n^3 + 3n^2 + 2n + 1$$

算法的时间复杂度: $O(n^3)$ **立方阶**

- 计算方法1: 由于是一个三重循环, 每个循环从1到n, 则总次数为: $n \times n \times n = n^3$ 故时间复杂度为 $T(n) = O(n^3)$ 。
- 计算方法2: 由于“乘法”运算是本例的基本操作, 故整个算法的执行时间与该基本操作 (乘法) 重复执行的次数 n^3 成正比, 故时间复杂度为 $T(n) = O(n^3)$ 。

时间复杂度计算

例5:分析以下程序段的时间复杂度

`i=1;` //语句频度为: 1

`while (i<=n)`

`i=i*2` //语句频度为: $f(n)$

因为: $2^{f(n)} \leq n$, 即: $f(n) \leq \log_2 n$, 取最大值 $f(n) = \log_2 n$, 则该程序的时间复杂度为: $T(n) = 1 + f(n) = 1 + \log_2 n = O(\log_2 n)$

时间复杂度计算

□ 算法中基本操作重复执行的次数随问题的输入数据集不同而不同

例6：在数组A[n]查找给定值K

```
(1)   i=n-1;  
(2)   while ( i >= 0 && A[i] != K )  
(3)       i=i-1;  
(4)   RETURN(i);
```

最好情况的时间复杂度 $T(n)=O(1)$

最坏情况的时间复杂度 $T(n)=O(n)$

平均时间复杂度：

所有可能的输入实例以等概率出现的情况

$$T(n) = (1 + 2 + \dots + n) / n$$

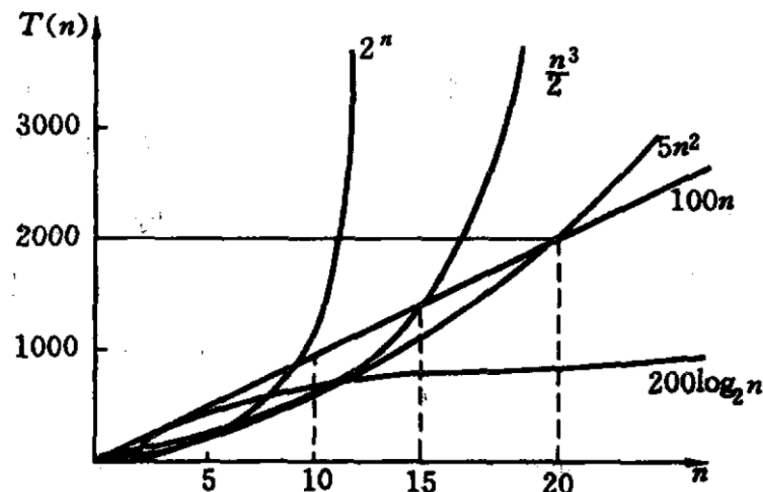
算法的时间复杂度： $O(n)$

时间复杂度按数量递增排列及增长率

- 最佳、最差、平均时间复杂度
 - 无法精确计算基本操作的执行次数（频度）时，只须求出其关于 n 的增长率
 - 与输入数据集有关，如冒泡程序，可考虑平均时间复杂度和最坏情况下的时间复杂度

1.4 算法和算法分析

$O(1)$	常量阶, 与 n 无关
$O(\log n)$	$\log n$ 阶
$O(n)$	n 阶
$O(n \log n)$	$n \log n$ 阶
$O(n^2)$	平方阶
$O(n^3)$	立方阶
$O(2^n)$	指数阶



常见复杂度: $O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3)$

指数复杂度: $O(n^k) < O(2^n) < O(n!) < O(n^n)$

●说明: 教材p17第2行: 除特别指明外, 均指最坏情况下的时间复杂度

时间复杂度按数量递增排列及增长率

表：时间复杂度和算法运行时间的关系

$\begin{matrix} T(n) \\ n \end{matrix}$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(n^5)$	$O(2^n)$	$O(n!)$
20	4.3us	20us	86.4us	400us	8ms	3.2s	1.05s	771世纪
40	5.3	40	213	1600	64ms	1.7min	12.7天	2.59×10^{32} 世纪
60	5.9	60	354	3600	216ms	13min	366世纪	2.64×10^{66} 世纪

时间复杂度按数量递增排列及增长率

结论:

- (1) 当 $f(n)$ 为对数函数、幂函数、或它们的乘积时，算法的运行时间是可以接受的，称这些算法是有效算法；当 $f(n)$ 为指数函数或阶乘函数时，算法的运行时间是不可接受的，称这些算法是无效的算法。
- (2) 随着 n 值的增大，增长速度各不相同， n 足够大时，存在下列关系：

对数函数 < 幂函数 < 指数函数

空间复杂度

- 类似于算法的时间复杂度，空间复杂度可以作为算法所需存储空间的量度。记作：

$$S(n) = O(f(n))$$

其中 n 为问题规模的大小。主要包括三个部分：

- (1) 输入数据所占用的空间。
- (2) 程序本身所占用的空间。
- (3) 辅助变量所占用的空间。

- 存储密度 $d = \text{数据本身存储量} / \text{实际所占存储量}$

正在答疑
